

BINV1051-B - Développement web : les bases du Back-End

Typescript-NodeJS-Express-Javascript

Jerome Plumat

2025

Table des matières

1	Introduction	5
1.1	Comment lire ce document	5
1.1.1	Résoudre les exercices du syllabus	5
1.2	Description du cours	6
1.2.1	Planning	6
1.2.2	Objectifs	6
1.2.3	Évaluations	7
1.2.4	Matériel de cours	7
1.3	Projet du cours	7
1.4	Prérequis	8
1.5	Visual Studio Code	8
1.5.1	Installation et configuration VSC	8
1.6	Conventions de nommage des variables	8
2	Le langage TypeScript	9
2.1	Introduction	9
2.2	Les bases	10
2.2.1	Les commentaires	11
2.3	Les variables et les types	12
2.3.1	Les variables de bases	12
2.3.2	Les types dans les variables	13
2.3.3	Les types de bases	13
2.3.4	Interfaces et types complexes	14
2.3.5	Opération sur les types	15
2.4	Les structures de contrôle	16
2.4.1	Les boucles	16
2.4.2	Les conditions	17
2.4.3	Les opérateurs logiques	17
2.5	Les fonctions	18
2.6	Les classes	19
2.6.1	Les classes fournies par TS	19
2.7	Les modules	20
3	NodeJS	23
3.1	npm	23
3.2	Express : un serveur NodeJS	25

4	Serveur RestFull	27
4.1	Résumé du chapitre	27
4.2	Introduction	27
4.3	RestFull serveur	28
4.3.1	RestFull API	28
4.3.2	Les ressources du serveur	29
4.4	URL et URI	29
4.4.1	Comment lire et écrire une URL	30
4.4.2	URL et la ressource	30
4.4.3	Paramètres	31
4.4.4	Query string	31
4.4.5	Résumé de l'URL	32
4.5	Requête	32
4.5.1	Méthodes HTTP et CRUD	32
4.5.2	Entêtes et Corps de la requête	33
4.5.3	Résumé de la requête	33
4.6	Réponses	34
4.6.1	Entêtes et Corps de la réponse	34
4.6.2	Codes de status	34
4.7	Données et JSON	35
5	Mise en pratique : Express	39
5.1	Serveur Express : les bases	39
5.2	Organisation d'une application Express	42
5.2.1	Controller	43
5.2.2	Modèle	44
5.2.3	Service	45
5.2.4	Le service d'accès base de données	48
5.2.5	Le service Logger	49
5.3	Les DTO	50
5.4	Convention de nommage	50
5.5	En bref	50
5.6	Exercices	50
5.7	Correction des exercices	50
6	Deux services particuliers	51
6.1	Logger	51
6.2	Logger as a service	52

Chapitre 1

Introduction

Le cours **BINV1051** "Développement web : bases" donné à la haute école Léonard de Vinci est scindé en trois parties. La première, effectuée durant le premier quadrimestre **BINV1051-A** présentait les bases de l'HTML. Les autres parties : **BINV1051-B** et **BINV1051-C**, abordées durant ce second quadrimestre, présentent les bases du développement web serveur avec le langage TypeScript et Javascript et le framework Node.js.

Ce document est le support syllabus du cours **BINV1051-B**, il vient en **complément** des diapositives fournies sur le site Moodle de la HE Vinci. Ce syllabus a pour objectif de présenter le contenu du cours, les objectifs, les évaluations, le matériel de cours et les liens utiles.

SECTION 1.1: COMMENT LIRE CE DOCUMENT

Différents codes couleurs sont utilisés dans ce document.

- **Texte en bleu** : Les mots en bleu sont des mots importants à retenir.
- les définitions sont présentées en **texte en vert foncé**.
- les noms de fichiers seront indiqués comme suit : `fichier.txt`
- toutes parties surlignées en jaune sont des **parties en travaux**.
- tous les mots faisant référence à un mot de code seront présentés dans un cadre gris clair. Par exemple la variable `doctor`.
- tous les URLs seront écrites ainsi `https://localhost:3000/doctors` ou le plus souvent de manière plus concise `/doctors`.

Il est à noter que tout le code présenté dans ce document est en anglais. L'anglais est la langue de développement par excellence. Il est donc important de s'habituer à lire et écrire du code en anglais. Cela vous permettra de mieux comprendre les documentations et les forums de développement (presque toujours en anglais). Enfin, dans un pays trilingue comme la Belgique, l'anglais est souvent la seule langue commune entre les développeurs, lors des réunions et dans l'échange d'emails.

1.1.1 RÉSOUDRE LES EXERCICES DU SYLLABUS

Explications

SECTION 1.2: DESCRIPTION DU COURS

Le cours BINV1051 apprendre les bases du développement web. Il est destiné aux étudiants de la première année du bachelier Informatique développement d'applications de la haute école Léonard de Vinci. Il est basé sur le langage TypeScript et Javascript ainsi que le framework Node.js.

Il est donné par les professeurs suivants :

- **Jérôme Plumat** : jerome.plumat@vinci.be
- **Stéphanie Ferneeuw** : stephanie.ferneeu@vinci.be
- **Sebastien Strebelle** : sebastien.strebelle@vinci.be
- **Jean Sébastien Derieuw** : jeansebastien.derieuw@vinci.be
- **Nicolas Burny** : jeansebastien.derieuw@vinci.be

La programmation web est un domaine en constante évolution. Les technologies web sont de plus en plus nombreuses et complexes. Ce cours a pour objectif de donner aux étudiants les bases nécessaires pour comprendre et développer celles-ci, toutefois, nous utiliserons des technologies récentes (notamment le langage TypeScript).

Les cours **BINV1051-B** et **BINV1051-C** sont particulièrement orientés vers le développement d'**API serveur**. C'est une tâche extrêmement importante dans le développement d'applications web. Les API permettent de communiquer entre les différentes parties d'applications (front-end, back-end, base de données, etc.). C'est également une source d'offres d'emploi importante dans le domaine de l'informatique depuis de nombreuses années. Enfin, les API permettent d'apprendre des concepts de base des serveurs.

1.2.1 PLANNING

Le cours **BINV1051-B** est réparti sur **6 semaines**. Il est composé d'une partie théorique mais aussi de la mise en pratique. A la fin des six semaines, le cours BINV1011 démarre et se poursuit sur les six dernière semaines du quadrimestre. En juin, deux évaluations certificatives finales sont prévues, une pour chaque partie du cours. L'examen du cours de BINV1051 se fera sur station, durera 2h et vous demandera d'implémenter des fonctionnalités sur un projet donné. Ce projet sera le même que celui utilisé durant les travaux pratiques.

Le cours **BINV1051-B** et **BINV1051-C** se composent de 9 heures de théorie et de 36 heures de travaux pratiques.

La partie théorique est à effectuer en autonomie à la maison **avant les séances pratique**, elle est composée de vidéos, de présentation et du présent document dénommé "syllabus". La partie travaux pratiques est à effectuer en présentiel lors de séances de 2 heures à raison de 3 séances par semaine.

1.2.2 OBJECTIFS

Les objectifs du cours sont les suivants :

- Coder dans le langage de programmation TypeScript et la production de code Javascript ;
- Mettre en application les concepts de base d'un serveur web en utilisant la librairie NodeJS ;
- Pouvoir mettre en place une API web respectant les standards REST ;
- Pouvoir mettre en application les concepts du "Happy Path" et la gestion des erreurs dans une API web ;

1.2.3 ÉVALUATIONS

L'évaluation s'effectuera en salle informatique et durera 2 heures. Elle vous demandera de produire du code répondant à certaines fonctionnalités et qualité de code. Vous serez évalué sur votre capacité à comprendre et à appliquer les concepts vus en cours dans un temps limité. Vous effectuerez des exercices de programmation en TypeScript similaires à ceux vus lors des dernières séances de travaux pratiques.

L'évaluation se fera sur un projet donné. Ce projet sera le même que celui utilisé durant les travaux pratiques. Vous devrez implémenter de nouvelles fonctionnalités au sein de ce projet. Les critères d'évaluation seront les suivants :

- La qualité du code ;
- La qualité de l'implémentation des fonctionnalités demandées ;
- La qualité de la gestion des erreurs ;
- La qualité de la gestion des logs ;
- La prise en considération de potentiels tests unitaires ;

1.2.4 MATÉRIEL DE COURS

Le cours dispose du matériel suivant :

- **Syllabus** : présent document, fourni en version PDF sur Moodle.
- **Diapositives** : résumant les concepts présentés dans ce syllabus, fournies en PDF sur Moodle au fil des semaines.
- **Vidéos** : présentant la mise en application des concepts théoriques du cours, disponibles sur Moodle au fil des semaines.
- **Les énoncés des travaux pratiques** : disponibles sur Moodle au fil des semaines.
- **Corrigés des travaux pratiques** : à chaque fin de semaine, les corrigés des travaux pratiques seront publiés, au début sur Moodle et par la suite sur **github**.

SECTION 1.3: PROJET DU COURS

La partie travaux pratiques sera basée, dès la deuxième séance, sur un projet qui nous accompagnera tout au long du cours. Ce projet est un serveur web qui permet de gérer un petit hôpital. Nous démarrerons sur une version de base et nous l'enrichirons au fur et à mesure des séances. C'est ce projet qui sera utilisé lors de l'évaluation certificative finale et il vous sera demandé d'implémenter de nouvelles fonctionnalités au sein de ce petit projet.

LIENS UTILES

Je vous recommande très fortement, lorsque vous rencontrez une interrogation ou une difficulté, de consulter les liens suivants :

- La documentation officielle de TypeScript : <https://www.typescriptlang.org/docs/>
- La documentation officielle de Node.js : <https://nodejs.org/en/docs/>
- Le site w3schools offre de très bonnes ressources pour le développement avec TypeScript et Node.js : <https://www.w3schools.com/nodejs/>

SECTION 1.4: PRÉREQUIS

Il n'y a pas de prérequis pour ce cours. Toutefois, il est fortement conseillé d'avoir suivi le cours **BINV1051-A** ou d'avoir des connaissances de base en HTML et CSS.

SECTION 1.5: VISUAL STUDIO CODE

Visual Studio Code (VSC) est un éditeur de code source développé par Microsoft pour Windows, Linux et MacOS. Il inclut la prise en charge du débogage, la mise en évidence de la syntaxe, la complétion intelligente du code, les snippets et la refactorisation du code. Il est également personnalisable, ce qui signifie que les utilisateurs peuvent changer le thème, les raccourcis clavier et les préférences. Il est gratuit et open-source, bien que l'éditeur lui-même soit sous une licence propriétaire.

Ce sera votre outils de travail principal durant ce cours. Il est donc important de bien le connaître et de l'utiliser correctement. Nous verrons, dans les sections suivantes, comment l'installer et le configurer pour qu'il soit le plus efficace possible.

Comme c'est un outils très complet, nous ne verrons que les éléments essentiels pour ce cours. Il est fortement conseillé de consulter la documentation officielle de Visual Studio Code pour en apprendre plus sur ses fonctionnalités <https://code.visualstudio.com/docs>.

1.5.1 INSTALLATION ET CONFIGURATION VSC

VSC permet l'installation de plugins pour étendre ses fonctionnalités. Nous utiliserons principalement les plugins suivants :

- **Prettier** : pour formater automatiquement notre code TypeScript.
- **REST Client** : pour tester nos API.

SECTION 1.6: CONVENTIONS DE NOMMAGE DES VARIABLES

Tout le code écrit dans le cadre de ce cours devra respecter les conventions de nommage suivantes :

- Le code doit être écrit en anglais.
- Les noms de variables doivent être écrits en **camelCase**. Par exemple, `myVariable`, `mySecondVariable`, etc.
- Toute variable faisant référence à une liste, un ensemble, tableau, etc. doit être suffixée par un `s`. Par exemple, `patients`, `dotors`, etc.
- Les paramètres d'entrée et de sortie des fonctions doivent toujours être typés explicitement. Par exemple, `function myFunction(param1: string, param2: number): void {...}`.
- Toutes les requêtes HTTP doivent contenir un code de retour (status code) et un message d'erreur explicite. Par exemple, `res.status(404).send('Not found')`.

Chapitre 2

Le langage TypeScript

Le chapitre ci-dessous donne une base et quelques concepts clés à la programmation TypeScript et NodeJS. Ces notions ici définies sont amenées à être approfondies et appliquées dans les vidéo et pratiquées tout au long des séances de travaux pratiques. Les différents professeurs vous invitent très chaleureusement à consulter par vous-même et en autonomie les différentes ressources extérieures fournies dans le syllabus pour résoudre les problèmes rencontrés ou pour approfondir vos connaissances.

SECTION 2.1: INTRODUCTION

Dans le cadre de ce cours, nous utiliserons le langage **TypeScript** (TS). Ce langage est une version moderne du langage Javascript (JS). La modernité de ce langage réside dans le fait qu'il est fortement typé. Cela permet de détecter des erreurs de programmation à la compilation plutôt qu'à l'exécution. Cela permet également une programmation plus facile avec le programme Visual Studio Cod (VSC).

Le code TS n'est pas directement exécutable par un navigateur. Il doit être transformé en code JS pour être exécuté. C'est ce que nous ferons en utilisant le **compilateur TypeScript**. Ce compilateur transformera le code TS en code JS. Ce code JS pourra alors être exécuté par un navigateur ou par NodeJS.

Le langage Javascript est un langage interprété. Il peut être interprété par un navigateur internet ou dans l'environnement NodeJS.

Historiquement, le JS a été développé pour être exécuté dans un navigateur. Il était utilisé pour rendre les pages web dynamiques et interactives. Toutefois, au fil des années, il fut de plus en plus utilisé pour créer des applications web, des applications mobiles, des applications de bureau, etc. Cela a conduit à la création de NodeJS.

NodeJS est un environnement d'exécution de Javascript en dehors d'un navigateur. Il permet d'exécuter des programmes Javascript en ligne de commande. Une des applications les plus connues de NodeJS est la création de serveurs web, généralement dans le framework Express.

À l'inverse du langage Javascript, TypeScript ne peut pas être exécuté directement par un navigateur ou NodeJS. **Il doit être transformé en Javascript avant d'être exécuté.** C'est ce que nous ferons en utilisant le compilateur TypeScript. **Nous écrirons donc du code TypeScript** que nous **transformerons** en **code Javascript** pour l'exécuter au travers de

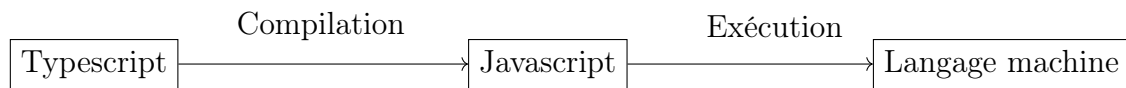


FIGURE 2.1 – Le cycle de vie d'un programme TypeScript

NodeJS. Le code JS ainsi produit par la compilation n'est pas destiné à être lu ou modifié par un humain. Il est destiné à être exécuté de manière efficace et sécurisée par un ordinateur. La figure 2.2 illustre ce processus de compilation.

Je vous invite à regarder la vidéo présente sur Moodle présentant le langage TS et son utilisation au sein de VSC. Cela vous permettra d'appliquer les concepts vus dans ce chapitre et expérimenter par vous même de façon à vous familiariser avec le langage.

De plus, vous devez télécharger les fichiers présents sur Moodle et les ouvrir dans VSC. Vous pourrez alors suivre les instructions données dans les fichiers pour les compiler et les exécuter. Nous assumerons que vous avez déjà importé ces fichiers et installer les dépendances (**npm install**) tel que présenté dans la vidéo.

Le langage TypeScript (TS) est un langage de programmation open-source développé par Microsoft. Il est une surcouche du langage Javascript (JS). Il existe différentes normes quand à la façon d'écrire le TS, le monde de l'entreprise a tendance à suivre la norme ECMAScript 6 (ou ES6), c'est également la norme que nous suivront dans ce cours.

Depuis des décénies le JS a montré de nombreuses qualités en tant que langage de programmation. La facilité d'écrire du code, la rapidité de développement, la facilité de débogage, la portabilité, la compatibilité avec les navigateurs web, etc. ont permis que celui-ci s'impose dans de très nombreux domaines. Il est reconnu comme l'un des langage les plus utilisés au monde.

Cependant, le JS a également montré des défauts. Parmi les défauts les plus connus, on peut citer l'absence de typage des variables, la gestion des erreurs à l'exécution, la gestion des dépendances, etc. Ceci a conduit l'industrie (et plus particulièrement Microsoft) à développer le TypeScript. *Le TS est un langage qui a but d'être transformé en JS pour ensuite être exécuté.* Il permet de corriger les défauts du JS tout en conservant ses qualités.

Voyons maintenant les bases du langage TS.

SECTION 2.2: LES BASES

Le langage TS est un langage de programmation **orienté objet**. Il permet de définir des variables, des fonctions, des classes, des interfaces, etc. Chaque instruction s'écrit sur une ligne et **se termine par un point-virgule**. Par exemple, un fichier `file.ts` contenant le code suivant :

```
1  const a = 1;
2  console.log(a);
```

Ce code définit une simple variable constante `a` et lui assigne la valeur 1. La deuxième ligne affiche la valeur de la variable `a` dans le terminal. Les fichiers TS ont une extension `.ts`. Pour exécuter un fichier TS, il faut le compiler en JS. Pour cela, il faut utiliser le compilateur TS : **tsc**. Par exemple, dans un terminal :

```
tsc file.ts
```

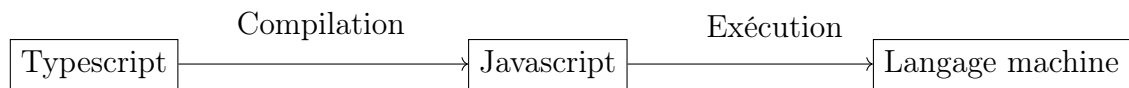


FIGURE 2.2 – Le cycle de vie d'un programme Typescript

Si aucune erreur n'est détectée, le compilateur génère un fichier `file.js` qui peut être exécuté avec NodeJS. Pour exécuter le fichier `file.js` avec NodeJS, il faut utiliser la commande suivante dans un terminal :

```
node file.js
```

Le code sera alors exécuté et le résultat affiché dans le terminal. Dans notre cas "1" sera affiché.

2.2.1 LES COMMENTAIRES

Les commentaires en TS sont similaires aux commentaires en Java. Il existe deux types de commentaires :

- Les commentaires sur une seule ligne qui commencent par `//`. Par exemple :

```
1 // this is a comment
```

- Les commentaires sur plusieurs lignes qui commencent par `/*` et se terminent par `*/`. Par exemple :

```
1 /*  
2     This is a comment  
3     on multiple lines  
4     */  
5
```

SECTION 2.3: LES VARIABLES ET LES TYPES

2.3.1 LES VARIABLES DE BASES

En TS, les variables sont déclarées avec le mot-clé `let` ou `const`. La différence entre les deux est que `let` permet de réaffecter une valeur à une variable alors que `const` ne le permet pas. Dans le second cas, la variable est assignée à une valeur constante. **Par défaut, il vous est demandé d'utiliser le mot clé `const` par défaut lorsque cela est possible.** Ainsi, les instructions suivantes sont valides :

```
1 let a = 1;
2 console.log(a);
3 const b = 'Hello';
4 const c = [1, 2, 3];
```

La ligne 1 crée une variable `a` et lui assigne la valeur 1. La ligne 2 affiche la valeur de la variable `a`. La ligne 3 crée une variable `b` et lui assigne la valeur `"Hello"`, la variable `b` est constante et nous ne pourrions pas changer la valeur de la chaîne de caractère. La ligne 4 crée une variable `c` et lui assigne comme valeur une liste. Cette liste, contient trois élément. Dans la ligne 4, seule la référence de la liste est constante, il est donc possible de modifier les éléments de la liste. Ainsi, les instructions suivantes sont valides :

```
1 const c = [1, 2, 3];
2 c[0] = 4;
3 c.push(5);
```

La ligne 2 modifie le premier élément de la liste, la ligne 3 ajoute un élément à la liste. Nous aurons donc au finale, une variable `c` qui contient les éléments 4, 2, 3 et 5. Tous de types `number`. Attention, à l'inverse, les instructions suivantes sont invalides :

```
1 // /\ this code is NOT valid
2 const c = [1, 2, 3];
3 c = [4, 5, 6];
4 const d = [1,3,4];
5 d = c;
```

Les ligne 2 et 4 tentent de réaffecter une valeur à une variable constante. Cela est impossible en TS et provoque une erreur visible dans le programme Visual Studio Code (VSC) ainsi que lors de la compilation

2.3.2 LES TYPES DANS LES VARIABLES

Lorsqu'on crée une variable, un type lui est attribué. Pour afficher le type dans le terminal, on peut utiliser le code suivant :

```
1 let a = 1;
2 console.log(typeof a);
```

L'exécution de ce code affichera `number` dans le terminal. Cela signifie que la variable `a` est de type `number`. Notons que null part dans la ligne 1 nous ne spécifions le type de la variable. Ce type est **automatiquement inféré** (*i.e.* déduit) par le compilateur.

Même si le compilateur peut déduire lui même le type, il est parfois important de lui spécifier. Il est possible de spécifier le type d'une variable en utilisant la syntaxe suivante :

```
1 let a: number = 1;
2 const b: string = 'Hello';
3 const c: number[] = [1, 2, 3];
```

Ces lignes sont en tout point équivalentes aux lignes présentée en 2.3.1. La différence est que le type de chaque variable est explicitement défini. Cela permet de mieux comprendre le code et de détecter des erreurs de programmation à la compilation.

Une fois qu'une variable a été déclarée, il est pas possible de lui assigner une valeur d'un autre type. Ainsi, le code suivant :

```
1 // /\ this code is NOT valid
2 let a: number = 1;
3 a = 'Hello';
```

Le code ci-dessous provoque une erreur à la compilation. L'erreur affichée par le compilateur `tsc` est la suivante : `Type 'string' is not assignable to type 'number'`. Cela signifie que le type `string` n'est pas assignable au type `number`.

Tout au long de ce cours, nous n'explicitons les types de toutes les variables. Toutefois, lorsque cela est nécessaire (et cela le sera souvent), nous spécifierons explicitement le type des variables. Cela permettra de mieux comprendre le code et de détecter des erreurs de programmation à la compilation. Nous reviendrons sur ce point à différents moments dans ce chapitre.

2.3.3 LES TYPES DE BASES

Les types de bases en TS sont les suivants :

boolean pour les valeurs booléennes (`true` et `false`), c'est l'équivalent du type `boolean` en Java. Exemple :

```
1 let a: boolean = true;
2 a = false;
```

number pour les nombres entiers ou à virgule flottante, c'est l'équivalent du type **double** en Java.

```
1 let a: number = 1;
2 a = 1.5;
```

string pour les chaînes de caractères, c'est l'équivalent du type **String** en Java. Attention, le type **string** s'écrit **sans majuscule** à l'inverse du Java. L'on peut définir des chaînes de caractères avec des guillemets simples ou doubles ainsi que des backticks (```). Les backticks permettent d'insérer des variables dans une chaîne de caractères.

```
1 let a: string = 'Hello';
2 let b: string = "World";
3 let c: string = `Hello ${b}`;
```

undefined et **null** : sont des types à part entière. Ils sont utilisés pour définir des variables qui n'ont pas de valeur (pour **undefined**) ou qui ont une valeur nulle (pour **null**). Par exemple :

2.3.4 INTERFACES ET TYPES COMPLEXES

Un des grands avantages de TS est la possibilité d'utiliser des types complexes et cela en utilisant des interfaces.

Attention à ne pas confondre la notion d'interface vue dans le cours d'APOO et la notion d'interface vue dans le cadre de ce cours dans le langage JAVA. Dans le cadre de ce cours et dans le cadre du langage TS, une **interface** est **un moyen de définir un type complexe**. Par exemple, nous pouvons définir un type **Doctor** comme suit :

```
1 interface Doctor {
2     name: string;
3     age: number;
4     speciality: string;
5 }
6 const doc: Doctor = {
7     name: 'John Doe',
8     age: 45,
9     speciality: 'Cardiologist'
10 }
```

Les quatres premières lignes permettent de définir le type **Doctor**, l'interface présente les trois propriétés du type **Doctor** chacune possède un nom (**name**, **age**, **speciality**) et un type (**string**, **number**, **string**). Les dernières lignes créent une variable **doc** de type **Doctor** et lui assigne une valeur. Cette valeur est un objet qui respecte la structure de l'interface **Doctor**.

A l'inverse, les instructions suivantes sont invalides lorsqu'on ne spécifie pas toutes les valeurs d'une variable :

```
1 // /\ this code is NOT valid
2 interface Doctor {
3   name: string;
4   age: number;
5   speciality : string;
6 }
7 const doc: Doctor = {
8   name: 'John Doe',
9   age: 45,
10  // missing the property speciality
11 }
```

Dans ce cas, le compilateur TS et le programme VSC affichera une erreur car la variable `doc` ne respecte pas la structure de l'interface `Doctor`..

Notons que dans les deux cas, nous précisons de manière dynamique le type de la variable `doc`. Il en sera toujours ainsi dans le cadre de cours : **toutes les variables avec un type complexe devront être explicitement typées**. Cela permet de mieux comprendre le code et de détecter des erreurs de programmation à la compilation.

De la même façon, il n'est pas possible de spécifier une valeur qui n'est pas dans l'interface :

```
1 // /\ this code is NOT valid
2 interface Doctor {
3   name: string;
4   age: number;
5   speciality : string;
6 }
7 const doc: Doctor = {
8   name: 'John Doe',
9   age: 45,
10  speciality : 'Cardiologist',
11  address: '1 rue de la paix'
12 }
```

De nouveau ici, le compilateur TS et le programme VSC affichera une erreur car la variable `doc` contient une propriété `address` qui n'est pas dans l'interface `Doctor`.

2.3.5 OPÉRATION SUR LES TYPES

Un des avantages d'un langage fortement typé comme TS est la possibilité de faire des opérations sur les types. Par exemple, il est possible de faire des unions de types, des intersections de types, des différences de types, etc. Par exemple :

```
1 let a : number | string;
2 a = 1;
3 a = 'Hello';
```

Ainsi la variable `a` peut prendre une valeur de type `number` ou de type `string`. Il est également possible de faire des intersections de types. Par exemple :

```
1 interface A {  
2   name: string;  
3   age: number;  
4 }  
5 interface B {  
6   name: string;  
7   speciality : string;  
8 }  
9 type C = A & B;  
10 const c: C = {  
11   name: 'John Doe',  
12   age: 45,  
13   speciality : 'Cardiologist'  
14 }
```

Ainsi la variable `c` est de type `C` qui est l'intersection des types `A` et `B`. La variable `c` doit donc respecter les propriétés des deux interfaces `A` et `B`.

Dans le cadre de notre cours, nous utiliserons principalement les unions de types. Nous verrons comment les utiliser dans les chapitres suivants et plus particulièrement en tant que retour de fonction.

SECTION 2.4: LES STRUCTURES DE CONTRÔLE

2.4.1 LES BOUCLES

Les boucles en TS sont similaires aux boucles qu'en Java. Nous avons les boucles `for`, `while` et `do while`. Par exemple :

```
1 for(let j = 0; j < 10; j++){  
2   console.log(j);  
3 }  
4 let i = 0;  
5 while(i < 10){  
6   console.log(i);  
7   i++;  
8 }  
9 i = 0;  
10 do{  
11   console.log(i);  
12   i++;  
13 }while(i < 10);
```

Dans le cadre de ce cours, nous utiliseront essentiellement la boucle `for` et `while` pour parcourir des listes.

2.4.2 LES CONDITIONS

De la même façon, les conditions en TS sont similaires aux conditions en Java. Nous avons les conditions `if`, `else if` et `else`. Par exemple :

```
1  const a = 1;
2  if (a === 1) {
3    console.log('a is equal to 1');
4  } else if (a === 2) {
5    console.log("a is equal to 2");
6  } else {
7    console.log('a is different from 1 and 2');
8  }
```

2.4.3 LES OPÉRATEURS LOGIQUES

Lorsqu'on souhaite faire des opérations logiques en TS, nous utilisons les opérateurs `&&` pour l'opération *et*, et `||` pour l'opérateur *ou*. Par exemple :

```
1  const a = 1;
2  const b = 2;
3  if(a === 1 && b === 2){
4    console.log('a is equal to 1 and b is equal to 2');
5  }
```

Dans cet exemple, la condition est vraie si et seulement si `a` est égal à 1 et `b` est égal à 2.

Notez immédiatement que en TS nous utiliserons l'opérateur `===` pour comparer deux valeurs. Cet opérateur compare la valeur et le type des deux valeurs. Cet opérateur (avec trois signes égal) est apparu avec le langage JS et est devenu la norme dans le monde du développement web.

L'opérateur de négation en TS est `!`. Par exemple :

```
1  const a = 1;
2  if(a !== 2){
3    console.log('a is different from 2');
4  }
```

SECTION 2.5: LES FONCTIONS

Il est possible de créer des fonctions en TS. Ces fonctions peuvent se trouver au sein d'une classe (nous en reparlerons dans la section suivante) ou d'un module (un fichier pouvant être importé, nous y reviendrons dans la section 2.7). Pour le moment, nous nous intéressons aux fonctions en dehors des classes et concentrons-nous sur la façon des les écrire.

Une fonction simple peut être définie comme suit :

```
1  /** Function performing the addition of two numbers
2   * @param a the first number
3   * @param b the second number
4   * @return the sum of a and b
5   */
6  function add(a: number, b: number): number {
7      return a + b;
8  }
9  console.log(add(1, 2)); // calls of the function
```

La première ligne contient la définition de la fonction `add`. Cette fonction prend deux paramètres de type `number` et retourne un résultat de type `number`. La quatrième ligne appelle la fonction `add` avec les paramètres 1 et 2 et affiche dans le terminal. La définition d'une fonction respecte la syntaxe suivante :

```
1  function myFunction(param1: type1, param2: type2, ...): returnType {
2      // the code of your function
3  }
```

Toutefois, il existe d'autres façons de définir une fonction en TS. Il est possible de définir une fonction anonyme et de l'assigner à une variable. Par exemple :

```
1  const add = function(a: number, b: number): number {
2      const c = a + b;
3  }
4  console.log(add(1, 2));
```

Notons que la syntaxe générale est la même que pour une fonction classique. La différence est que la fonction n'a pas de nom et est assignée à une variable. Cette variable peut être utilisée comme une fonction classique. Enfin, il existe une dernière façon de définir une fonction en TS. Il est possible d'utiliser une fonction fléchée. Par exemple :

```
1  const add = (a: number, b: number): number => {
2      return a + b;
3  }
4  console.log(add(1, 2));
```

Dans ce dernier cas, la fonction est définie de façon plus concise tout en restant anonyme. La syntaxe est plus simple et plus lisible. Cette syntaxe est souvent utilisée pour les fonctions simples ou passées en paramètre d'autres fonctions.

Dans le cadre de ce cours, **lorsqu'on déclare une fonction, il sera demandé de toujours spécifier le type de retour de la fonction et le type de tous les paramètres de la fonction**. Cela permet de mieux comprendre le code et de détecter des erreurs de programmation à la compilation.

SECTION 2.6: LES CLASSES

Les classes dans le langage TS sont similaires aux classes dans le langage Java. Elles permettent de définir des objets avec des propriétés et des méthodes. Par exemple :

```
1 class Rectangle {
2   public length: number;
3   public width: number;
4   public color: string;
5   /** Constructeur */
6   constructor(length: number, width: number, color: string){
7     this.length = length;
8     this.width = width;
9     this.color = color;
10  }
11  /** Functions */
12  public print(): string {
13    return `Rectangle of length ${this.length}, width ${this.width} and color ${this.color}`;
14  }
15 }
16 const rect = new Rectangle(10, 5, 'blue');
17 rect.print();
```

Nous commençons par définir la classe `Rectangle` avec trois attributs (ou propriétés) (`length`, `width`, `string`), notez que l'on spécifie à chaque fois le type de ces attributs. Nous définissons ensuite un constructeur à l'aide du mot clé `constructor` qui est une fonction qui prend trois paramètres et initialise les propriétés de la classe. Enfin, nous définissons une fonction (méthode) `print` qui affiche un message dans le terminal. Nous créons ensuite une instance de la classe `Rectangle` à l'aide du mots clé `new` et assignons cette nouvelle instance à la variable `rect`. Enfin, nous appelons la méthode `print` de l'objet `rect`.

2.6.1 LES CLASSES FOURNIE PAR TS

LES STRING

La classe `string` est une classe fournie par TS. Elle permet de manipuler des chaînes de caractères. Par exemple :

```
1 const a = 'Hello';
2 console.log(a.length); // affiche 5
3 console.log(a.toUpperCase()); // affiche HELLO
```

Plus d'information sur l'API de la classe `string` peut être trouvée sur le site officiel de TS, <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#string>.

LES LISTES

La classe `Array` est une classe fournie par TS. Elle permet de manipuler des listes. Par exemple :

```
1 const a = [1, 2, 3];
2 console.log(a.length); // affiche 3
3 console.log(a[0]); // affiche 1
4 a.push(4);
5 console.log(a); // affiche [1, 2, 3, 4]
```

Plus d'information sur l'API de la classe `Array` peut être trouvée à l'adresse <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#array>.

L'OBJET REQUEST ET RESPONSE

Une fois notre serveur en place, nous utiliserons les objets `Request` et `Response` fournis par le module `express`. Ces objets permettent de lire les paramètres d'une requête HTTP et de renvoyer une réponse HTTP. Par exemple :

```
1 import { Request, Response } from 'express';
2 app.get('/hello', (req: Request, res: Response) => {
3   // extract the parameter name from the request
4   const name = req.query.name;
5   // extract the body of the request
6   const body = req.body;
7   // send a response
8   res.send('Hello World');
9 });
```

SECTION 2.7: LES MODULES

Les modules ont une place importante en TS. Ils permettent de découper le code en plusieurs fichiers et de les réutiliser. Un module est un fichier qui contient du code TS. Il est possible d'exporter des éléments d'un module pour les réutiliser dans un autre module. Par exemple :

```
1 // file1.ts
2 export function add(a: number, b: number): number {
3   return a + b;
4 }
5 // file2.ts
6 import { add } from './file1';
7 console.log(add(1, 2));
```

Dans notre exemple, nous avons deux fichiers `file1.ts` et `file2.ts`. Le premier fichier contient une fonction `add` qui prend deux paramètres et retourne la somme de ces deux paramètres. Notez le mot clé `export` qui permet d'exporter la fonction `add`. Si ce mot clé n'est pas mis devant la fonction (ou la classe, ou tout autre élément) alors cette fonction ne pourra pas être importée dans un autre module. Le deuxième fichier importe la fonction `add` du premier fichier,

pour cela, on utilise le mot clé `import` suivi du nom de la fonction que l'on souhaite importer et du **chemin** du fichier qui contient sa définition. Enfin, on appelle la fonction `add` avec les paramètres 1 et 2.

De plus, lorsqu'on souhaite utiliser des objets complexes déjà fournis par les modules de NodeJS, il faut les importer. Par exemple, pour utiliser les objets `Request` et `Response` fournis par le module `express`, il faut les importer. Par exemple :

```
1 import { Request, Response } from 'express';
```

Cette commande doit se trouver en début de fichier.

Nous utiliserons abondamment ce découpage en modules dans le cadre de ce cours. Il est donc important de bien comprendre cette notion.

Chapitre 3

NodeJS

L'environnement NodeJS est un environnement d'exécution de Javascript en dehors d'un navigateur. Il permet d'exécuter des programmes JS en ligne de commande depuis un terminal. NodeJS est basé sur le moteur V8 de Google Chrome. Il est open-source et multi-plateforme. Il est très utilisé dans le monde de l'entreprise pour développer des applications web, des applications mobiles, des applications de bureau, etc.

Nous utiliserons ici NodeJS pour exécuter du code Javascript produit à partir de notre code TypeScript. Le code que nous produirons aura pour objectif de créer un serveur web. Ce serveur web sera capable de répondre à des requêtes HTTP et de fournir des ressources à la demande. Tout au long des prochains chapitres, nous verrons en détails comment créer un serveur web simple, comment répondre à des requêtes HTTP, comment lire des paramètres de requêtes, etc. Cette section explique les différents éléments d'un serveur NodeJS et comment les utiliser.

SECTION 3.1: NPM

Le programme **npm** est un gestionnaire de paquets pour NodeJS. Il permet de gérer les dépendances d'un projet NodeJS. Il permet d'installer des paquets, de les mettre à jour, de les supprimer, etc. Il permet également d'exécuter des commandes spécifiques à un projet NodeJS. Ces commandes sont définies dans le fichier **package.json**. Voici un exemple de fichier **package.json** :

```
1  {
2    "name": "my-server-example",
3    "version": "1.0.0",
4    "description": "A simple json server",
5    "main": "exec/main.js",
6    "author": "Plumat Jerome",
7    "scripts": {
8      "start": "node exec/main.js"
9      "dev": "tsc main.ts --watch && node exec/main.js"
10   },
11   "dependencies": {
12     "express": "^4.17.1"
13   }
```

14 }

Ce fichier contient les informations essentielles du projet. Il présente le nom du projet, sa version, sa description, l’auteur principal, etc. Il contient également des dépendances du projet. Ces dépendances sont des paquets NodeJS qui sont nécessaires pour le projet. Par exemple, dans le fichier `package.json` 3.1, le projet dépend du paquet `express` en version `4.17.1`.

Pour installer les dépendances d’un projet NodeJS, il faut utiliser la commande suivante dans un terminal :

```
npm install
```

Le fichier `package.json` contient également différentes commandes facilitant le développement du projet. Par exemple, la commande `dev` permet de compiler le code TS en JS et de lancer le serveur NodeJS. La commande `start` permet de lancer le serveur NodeJS. Les deux commandes font référence à un fichier JS qui est le point d’entrée du serveur. Ce fichier est défini dans le champ `main` du fichier `package.json`. Il s’agit du fichier `main.js` dans le répertoire `exec`.

Ainsi, pour démarrer votre serveur NodeJS, il est possible d’utiliser la commande suivante dans un terminal (voir sa définition dans le fichier `package.json` 3.1) :

```
npm run dev
```

Listing 3.1 – la commande classique permettant de travailler efficacement

Cela aura pour effet de démarrer votre serveur en mode développement au travers du fichier point d’entrée : `main.js`.

Dans le cadre de ce cours, et à partir de la seconde semaine, vous aurez accès un code de base d’un serveur représentant une version simple d’un hôpital. Ce serveur sera enrichi au fur et à mesure des séances. Il vous sera demandé de compléter ce serveur pour ajouter de nouvelles fonctionnalités. Ce serveur sera fourni avec des fichiers `package.json` et `tsconfig.json` préconfigurés. Vous n’aurez qu’à exécuter la commande `npm run dev` pour démarrer le serveur.

SECTION 3.2: EXPRESS : UN SERVEUR NODEJS

Le premier fichier de configuration de notre environnement de développement est le fichier `package.json`. Un deuxième fichier important de notre environnement de travail est `tsconfig.json`. Ce fichier contient les options de compilation du compilateur TypeScript. Il permet de définir les fichiers à compiler, le répertoire de sortie, les options de compilation, etc. Voici un exemple de fichier `tsconfig.json` :

```
1 {
2   "compilerOptions": {
3     "target": "es5",
4     "module": "commonjs",
5     "outDir": "exec",
6     "strict": true,
7     "esModuleInterop": true
8   }
9 }
```

Notez en particulier l'option `outDir` qui définit le répertoire de sortie des fichiers JS. Dans notre cas, les fichiers JS seront placés dans le répertoire `exec`. Si ce répertoire n'est pas déjà présent dans votre dossier, il sera créé.

Enfin, le troisième fichier est le fichier `main.ts`. Ce fichier contient le code permettant de lancer le serveur NodeJS. Voici un exemple de fichier `main.ts` :

```
1 // import the app configuration
2 import app from 'app';
3 app.listen(3000, () => {
4   console.log("Server is running on port 3000");
5 });
```

Listing 3.2 – Exemple de fichier `main.ts`

Ce fichier TS n'est pas directement exécutable, nous l'avons dit, il nous faut d'abord le transformer en fichier JS. Pour cela, nous utilisons le compilateur TS. Pour compiler le fichier `main.ts` avec la commande suivante dans un terminal :

```
1 tsc main.ts
```

Cela aura pour effet de générer le fichier `main.js` dans le répertoire `exec` (voir fichier `tsconfig`). Finalement, ce nouveau fichier peut être exécuté avec NodeJS avec la commande suivante dans un terminal :

```
1 node main.js
```

Cette commande demande à l'environnement NodeJS d'exécuter le fichier `main.js`. Si aucune erreur n'est rencontrée, votre terminal affichera le message `Server is running on port 3000`. Vous pourrez alors ouvrir un navigateur web et taper l'adresse `http://localhost:3000` pour voir le message `Hello World!`. Votre premier serveur fonctionne !

À ce stade, si vous souhaitez modifier votre fichier `main.ts`, vous devrez terminer votre serveur, modifier le fichier `main.ts`, recompiler ce fichier en fichier `main.js` et redémarrer le serveur NodeJS. Pour éviter cela, il est possible d'utiliser le mode `watch` du compilateur TS. Pour cela, il faut utiliser la commande suivante dans un terminal :

```
1 tsc main.ts --watch
```

Ainsi, tous changements que vous ferez dans le fichier `main.ts` seront **automatiquement compilés** et le fichier `main.js` sera mis à jour et votre serveur redémarré. Tout cela sans intervention de votre part. Cela vous permettra de gagner du temps et de vous concentrer sur le développement de votre serveur.

Chapitre 4

Serveur RestFull

Ce chapitre présente les bases théoriques relatives à un serveur RestFull. Il présente les concepts de base de l'API RestFull, les méthodes HTTP, les codes de status, les requêtes et réponses, les URL et URI, les données et le format JSON.

Dans le chapitre suivant, nous mettrons ces notions en pratique en développant un serveur RestFull en utilisant le framework Express.js.

SECTION 4.1: RÉSUMÉ DU CHAPITRE

Serveur

URL

Requête

Réponse

SECTION 4.2: INTRODUCTION

Un **serveur** est un programme informatique qui fournit des services à d'autres programmes informatiques. On dira qu'un serveur répond à des **clients**. Ces clients peuvent être des programmes informatiques, des applications web, des applications mobiles, des pages HTML, etc. Un serveur est un programme qui tourne en permanence et qui attend des requêtes de clients. Lorsqu'un client envoie une **requête**, le serveur la traite et renvoie une **réponse** dans le plus bref délai. Un serveur peut traiter plusieurs requêtes en même temps. Il est donc



FIGURE 4.1 – Serveur, client, requête et réponse.

capable de gérer plusieurs clients simultanément.

Les services fournis par un serveur peuvent être très variés : stockage de fichiers, impression, accès à des bases de données, relais entre deux environnements, etc. Un **serveur web** est un serveur qui fournit des services web. Il peut fournir des pages web, des données, des fichiers, des services, etc. Un serveur **RestFull** est un serveur web qui fournit des services web RestFull. Un **service web RestFull** est un service web qui respecte les contraintes de l'architecture RestFull.

On dira qu'un client envoie une **requête** au serveur qui lui répond en lui renvoyant une **réponse**. Une **requête** est un message envoyé par le client au serveur. Une **réponse** est un message envoyé par le serveur au client. Une réponse peut être "positive" (*i.e* tout c'est bien passé) ou "négative" (*i.e.* une erreur c'est produite durant le traitement). Une requête et une réponse sont composées de plusieurs éléments : une **méthode**, une **URL**, des **entêtes** et un **corps**. La méthode est le type de requête (GET, POST, PUT, DELETE, etc.).

SECTION 4.3: RESTFULL SERVEUR

Il existe différentes façons de concevoir un serveur web. L'**architecture RestFull** est une architecture qui définit des contraintes pour concevoir un serveur web. Un serveur RestFull est un serveur web qui respecte ces contraintes. Parmi toutes celles-ci, les principales sont :

- **Client-serveur** : l'architecture RestFull est basée sur le principe de la **séparation des préoccupations**. Le client et le serveur sont deux entités distinctes. **Le client et le serveur sont indépendants l'un de l'autre.**
- **Sans état** (*stateless*) : l'architecture RestFull est basée sur le principe de la **transparence de l'état**. Le serveur ne garde pas de trace de l'état du client. Chaque requête est traitée de manière indépendante. Dit autrement, le serveur ne garde pas de trace de l'état du client entre deux requêtes (cela lui demanderait beaucoup trop de ressources si de nombreux clients contactent le client). **Le serveur ne garde pas de session.**
- **Cache** : l'architecture RestFull repose également sur le principe de la **mise en cache**. Le serveur peut envoyer des entêtes de cache au client pour lui indiquer si la réponse peut être mise en cache ou non. Le client peut mettre en cache la réponse pour éviter de la demander à nouveau au serveur. **Le cache permet d'améliorer les performances du serveur.**
- **Interface uniforme** : l'architecture RestFull est enfin basée sur le principe de l'**interface uniforme** (API). Le serveur expose une interface uniforme pour toutes les ressources. Toutes les ressources sont accessibles de la même manière au travers d'URL.

Le serveur que nous développerons dans ce cours respectera les contraintes listées ci-dessus. Il sera basé sur le framework **Express**. Expresss est un framework Node.js qui permet de créer des serveurs web RestFull. Express est très populaire pour le développement de serveurs web : il est simple à utiliser et très puissant. Il permet de créer des serveurs très rapidement et il est utilisé par l'industrie depuis plusieurs années.

4.3.1 RESTFULL API

Une **API** est un acronyme pour **Application Programming Interface**. Une API est un ensemble de règles et de conventions qui permettent de communiquer avec un programme informatique (dans notre cas, un serveur). On dit souvent qu'une API est un contrat entre deux

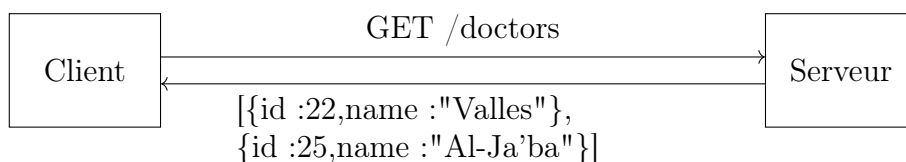


FIGURE 4.2 – Requête pour accéder à la ressource `doctors` avec la méthode `GET` sur le chemin `/doctors`. Le serveur répond en envoyant une liste d'objets contenant les informations des docteurs (ici, il y a deux docteurs avec les propriétés `id` et `name`).

acteurs l'appelant (dans notre cas le client) et l'appelé (ici le serveur). L'API définit les règles pour envoyer des requêtes correctes et quelles seront les réponses en retour.

L'API définit tous les éléments de la requête : les méthodes HTTP à utiliser, les URL à appeler, les entêtes à ajouter, et les éléments de la réponse : code de retour, les corps à envoyer, etc. Le client doit respecter les règles de l'API pour communiquer correctement avec le serveur.

4.3.2 LES RESSOURCES DU SERVEUR

Un serveur met à disposition de ses clients différentes ressources selon le contexte. Une **ressource** est une information ou un service fourni par le serveur. Une ressource peut être une page web, un fichier, une image, un service, etc. Une ressource est identifiée par son URL.

Dans le cadre de notre projet, notre serveur simulera un serveur d'un petit hôpital et mettra à disposition des **ressources** relatives à des patients, des docteurs, des rendez-vous, des examens médicaux, etc. Le client pourra formuler des requêtes pour accéder à ces ressources au travers d'URL. Par exemple, la figure 4.2 illustre un exemple de requête sur la ressource `doctors`. Le client effectuera un traitement sur base de réponse, par exemple : afficher la liste des docteurs sur une page web.

SECTION 4.4: URL ET URI

On appelle **URL** (Uniform Resource Locator) l'adresse d'une ressource sur le web. C'est grâce à cette adresse qu'un client peut formuler sa requête et ainsi accéder à un service ou une ressource que le serveur met à disposition. L'URL est composée de plusieurs parties, retenons quelques essentielles :

- **protocole** : `http`, `https`, `ftp`, etc.
- **type de requête** : `GET`, `POST`, `PUT`, `DELETE`, etc.
- **nom de domaine** : `www.google.com`, `www.facebook.com`, etc.
- **chemin** : `/`, `/index.html`, `/images/photo.jpg`, etc.
- **port** : `80`, `443`, `21`, `3000`, etc.
- **paramètres** : `?param1=valeur1¶m2=valeur2`, etc.

Dans la vie de tous les jours, nous utilisons des URLs pour accéder à des sites web. Nous tapons une URL dans la barre d'adresse de notre navigateur pour accéder à un site web. Au travers de lien hypertexte présents sur les pages web, nous formulons d'autres requêtes pour accéder à d'autres ressources. De la même façon, un programme informatique contactant un serveur le fera aussi à travers des URLs pour accéder à des ressources.

Une **URI** (Uniform Resource Identifier) est une chaîne de caractères qui identifie une ressource sur le web - sans informer sur comment y accéder. Une URI est plus générale qu'une URL. Toutes les URLs sont des URIs, mais toutes les URIs ne sont pas des URLs. Une URI peut identifier une ressource sans préciser comment y accéder alors qu'une URL identifie une ressource et précise comment y accéder.

Dans le cadre de ce cours, nous utiliserons essentiellement la notion d'URL.

4.4.1 COMMENT LIRE ET ÉCRIRE UNE URL

Lorsque nous navigons dans la vie de tous les jours, nous sommes habitué à des URLs "simples". Par exemple : `www.vinci.be`. On y voit le **nom de l'hôte** qui est ici "`www.vinci.be`". Cependant, il faudrait y rajouter le protocole `http` ou `https` (généralement complétée automatiquement par notre navigateur internet). Notre URL devient alors `http://www.vinci.be`. On peut également spécifier le port sur lequel le serveur écoute. Par exemple, dans notre cas, le serveur écoute sur le port 443 (et non sur le port 80, qui est le port par défaut), l'URL devient `http://www.vinci.be:443`. Enfin, on peut spécifier un chemin pour accéder à une ressource en particulier. Par exemple, si on veut accéder à la page d'accueil en anglais, l'URL devient `http://www.vinci.be:443/en`.

L'on remarque qu'une URL est composée de plusieurs éléments.

`http ://www.vinci.be : 443 / en`

Bien entendu, le serveur sur lequel nous allons travailler sera lancé sur notre machine locale et non sur une machine serveur externe. Le monde informatique a une convention pour désigner notre machine locale. Notre machine locale est désignée par le nom `localhost`. L'URL sera donc `http://localhost`. Si nous entrons cette URL dans un navigateur ou tout autre outils permettant d'effectuer des requêtes HTTP, nous accéderons à notre serveur local. Toutefois, notre serveur fonctionnera sur un port particulier, par défaut, et dans le cadre de ce cours, ce port est le port **3000**. Choix classique dans notre environnement Node.js. Notre adresse complète devient donc `http://localhost:3000`.

4.4.2 URL ET LA RESSOURCE

Les URLs que nous utilisons dans la vie de tous les jours (`www.vinci.be`, `www.moodle.vinci.be`, `www.linux.org`, etc.) font généralement référence à un page internet qui est envoyée et affichée dans le navigateur internet.

Dans le cadre de notre serveur, les URLs que nous utiliserons feront référence à des ressources. Par exemple, l'URL `http://localhost:3000/doctors` fera référence à la ressource

doctors. Le premier élément qui suit immédiatement le nom de domaine est le nom de la ressource que le client souhaite accéder.

Lorsqu'un client contactera le serveur avec cette URL, le serveur renverra la liste des docteurs. De la même façon, l'URL `http://localhost:3000/patients` fera référence à la ressource `patients`. Le serveur renverra la liste des patients.

4.4.3 PARAMÈTRES

Il est possible d'ajouter des **paramètres** à une URL pour préciser la ressource que l'on souhaite accéder. Les paramètres sont ajoutés à l'URL juste après le nom de la ressource. Par exemple, l'url `http://localhost:3000/doctors/1` fera référence au docteur avec l'identifiant 1, de la même façon, l'url `http://localhost:3000/doctors/2` fera référence au docteur avec l'identifiant 2.

Le paramètre permet de spécifier une ressource particulière. Le docteur avec l'id 1 ou 2.

4.4.4 QUERY STRING

La **query string** est un élément optionnel de l'URL. Elle est toujours le dernier élément de l'URL. Elle est utilisée pour **filtrer** la ressource demandée.

Elle se trouve toujours derrière le `?`. La query string est composée de plusieurs paramètres séparés par le symbole `&`. Par exemple, l'URL `http://localhost:3000/doctors?speciality=cardiologist&age=50`. Cette URL demande la liste des docteurs qui sont cardiologues et qui ont 50 ans. Elle encode deux clés `speciality` et `age` et deux valeurs ("cardiologist" et "50") au serveur. Le serveur utilisera ces paramètres pour filtrer la liste des docteurs.

4.4.5 RÉSUMÉ DE L'URL

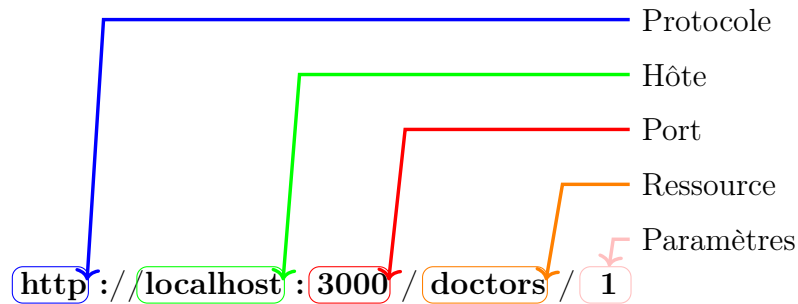


FIGURE 4.3 – Description d'une URL.

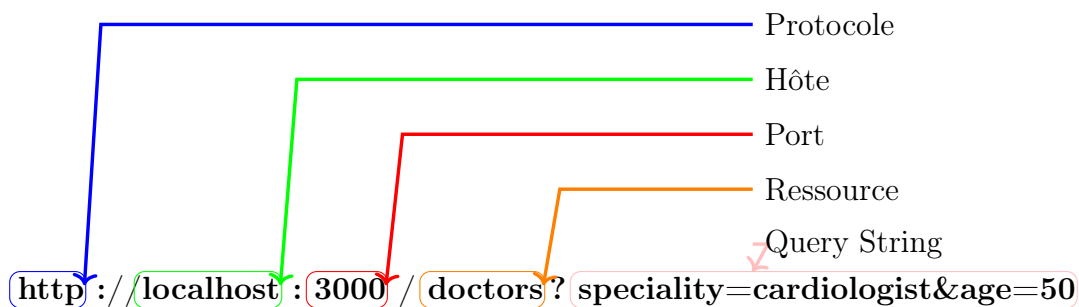


FIGURE 4.4 – Description d'une URL avec une query.

SECTION 4.5: REQUÊTE

Regardons maintenant en détails la requête qu'un client envoie à un serveur pour accéder à une ressource. Une requête est composée de plusieurs éléments que nous allons détailler ci-dessous.

4.5.1 MÉTHODES HTTP ET CRUD

Lorsqu'un **client** envoie une **requête** à un **serveur** au travers d'un **chemin** pour **accéder à une ressource**, il doit préciser la méthode HTTP à utiliser. La **méthode HTTP** est un mot-clé qui indique au serveur quelle action effectuer sur la ressource. Les méthodes HTTP les plus courantes sont les suivantes :

- **GET** : cette méthode est utilisée pour **lire** une ressource sans modifier celle-ci.
- **POST** : cette méthode est utilisée pour **créer** une nouvelle ressource.
- **DELETE** : cette méthode est utilisée pour **supprimer** une ressource. Attention, cette ressource ne sera plus accessible par la suite.
- **PUT** : cette méthode est utilisée pour **modifier** une ressource **ou** créer une ressource si celle-ci n'existe pas.
- **PATCH** : cette méthode est utilisée pour **modifier** une ressource **partiellement**. Si la ressource n'existe pas, une erreur sera produite.

Exemple Un chemin `/doctors` utilisé avec la méthode **GET** permet de **lire** la liste des docteurs. le même chemin `/doctors` utilisé avec la méthode **POST** permet de **créer** un nouveau docteur. **On peut donc contacter la même URL avec différentes méthodes.**

CRUD

On attache généralement aux différents méthodes HTTP des actions CRUD. **CRUD** est un acronyme pour **Create, Read, Update, Delete**. Ces actions correspondent aux méthodes HTTP suivantes :

- **Create** : POST
- **Read** : GET
- **Update** : PUT et PATCH
- **Delete** : DELETE

4.5.2 ENTÊTES ET CORPS DE LA REQUÊTE

TODO

Selon le type de requête, les paramètres peuvent être envoyés de différentes manières :

- **GET** et **DELETE** : les paramètres sont envoyés dans l'URL.
- **POST**, **PUT** et **PATCH** : les paramètres sont envoyés dans le corps de la requête.

PARAMÈTRES DANS L'URL

Lorsqu'on utilise les méthodes **GET** et **DELETE**, les paramètres sont encodés directement dans l'URL. Les paramètres sont envoyés après le nom de la ressource et sont séparés par un point d'interrogation (`?`). Les paramètres sont de la forme

```
param1=valeur1&param2=valeur2&param3=valeur3.
```

Par exemple, l'URL `http://localhost:3000/doctors?id=22&name=Valles` envoie deux paramètres `id` et `name` au serveur. Chaque paramètre a une valeur, ici le paramètre `id` a la valeur `22` et la paramètre `name` a la valeur `"Valles"`. Le serveur utilise alors ces paramètres pour traiter la requête.

PARAMÈTRES DANS LE CORPS DE LA REQUÊTE

Si les actions demandent de nombreux paramètres ou des paramètres complexes, il est préférable de les envoyer dans le corps de la requête. Les paramètres sont alors envoyés dans le corps de la requête (au format JSON mais nous y reviendrons à la section 4.7). Les méthodes liées à la modification ou la création d'une ressource (**POST**, **PUT** et **PATCH**) utilisent des paramètres complexes et passent donc ces paramètres dans le corps de la requête.

4.5.3 RÉSUMÉ DE LA REQUÊTE

La requête est composée de plusieurs éléments que nous avons détaillés ci-dessus. Les principaux éléments d'une requête sont les suivants :

1. l'URL : l'adresse utilisée pour contacter le serveur, désignant ainsi la ressource à laquelle on souhaite accéder.

- le nom de domaine : `www.vinci.be`.
 - le nom de la ressource : `/doctors`.
 - les paramètres : tout ce qui se trouve après le symbole `?` `id=22&name=Valles`.
2. la méthode HTTP utilisée pour contacter l'URL : GET, POST, PUT, DELETE ; celles-ci sont liées aux actions CRUD : Create, Read, Update, Delete.
 3. le corps de la requête contient les informations envoyées au serveur et forment une partie des paramètres de la requête.

SECTION 4.6: RÉPONSES

Regardons maintenant les différents éléments d'une réponse envoyée par un serveur à un client. La réponse est composée de plusieurs éléments que nous allons détailler ci-dessous. Ces différents éléments permettent au client de savoir si la requête a été traitée correctement ou non et de récupérer les ressources demandées si aucune erreur n'a été détectée.

4.6.1 ENTÊTES ET CORPS DE LA RÉPONSE

De la même façon que la requête est scindée en entêtes et corps, la réponse est également scindée en deux parties aux rôles identiques. Les **entêtes** contiennent des informations supplémentaires sur la réponse. Les entêtes peuvent contenir des informations sur le type de contenu, la taille du contenu, la date de la réponse, etc. Le **corps** de la réponse contient les ressources demandées par le client. Le corps peut contenir des données au format JSON, XML, HTML, etc. Si une erreur se produit l'entête contiendra un code indiquant l'erreur et le corps pourra être vide ou contenir plus d'information sur cette erreur.

4.6.2 CODES DE STATUS

le code de status indique si la requête a été traitée correctement ou non. Il existe de nombreux codes de status. Les codes de status les plus courants sont les suivants :

- **200** : *OK*, la requête a été traitée correctement.
- **201** : *Created*, la ressource a été créée.
- **204** : *No Content*, la requête a été traitée correctement mais il n'y a pas de contenu à renvoyer.
- **400** : *Bad Request*, la requête est incorrecte.
- **401** : *Unauthorized*, l'utilisateur n'est pas autorisé à accéder à la ressource.
- **403** : *Forbidden*, l'accès à la ressource est interdit.
- **404** : *Not Found*, la ressource n'a pas été trouvée.
- **500** : *Internal Server Error*, une erreur interne s'est produite sur le serveur.

Le code de status se trouve dans les entêtes de la réponse. Il permet au client de savoir si la requête a été traitée correctement ou non sans lire tout le contenu de la réponse. **Commencer par observer le code de status d'une réponse permet de gagner du temps et signe de bonne pratique lors de travail sur un serveur.** Il est important de connaître les principaux code d'erreur.

screen shot de FF avec les codes de status ?

SECTION 4.7: DONNÉES ET JSON

La réponse d'une requête peut contenir des informations liées à une ressources spécifiques (par exemple la ressource `doctors`). De la même façon, une requête demandant de créer une ressource doit contenir certaines informations pour que le serveur puisse mener à bien cette tâche.

Lorsqu'on échange des données entre éléments informatiques, il est important de définir un format commun pour échanger des données de façon structurée. Il existe de nombreux formats permettant d'échanger des informations (par exemple : XML, CSV, JSON, etc.).

Le format **JSON** (JavaScript Object Notation) est un format de données très populaire pour échanger des données entre éléments informatiques. JSON est un format de données simple et léger. Il est également facile à lire et à écrire pour les humains et facile à analyser et à générer pour les machines. JSON est basé sur un ensemble de paires clé-valeur. Si les données JSON sont stockées, elles le sont dans des fichiers avec l'extension **.json**.

Le format JSON impose des règles strictes pour écrire des données. Les données JSON sont composées de **paires clé-valeur**. Une paire clé-valeur est composée d'une clé et d'une valeur. La clé est toujours une chaîne de caractères qui identifie la valeur. La valeur peut être un nombre, une chaîne de caractères, un booléen, un tableau, un objet, etc. Les valeurs sont séparées par des virgules. Les paires clé-valeur sont séparées par des deux-points. Les objets sont délimités par des accolades. Les tableaux sont délimités par des crochets. Par exemple :

```
1 {
2   "id": 22,
3   "name": "Valles",
4   "speciality": "Cardiologue"
5 }
```

Ce code JSON représente un objet avec trois paires clé-valeur. L'objet JSON commence et se termine par des accolades (`{` et `}`). La clé `id` a pour valeur 22 (de type `number`), la clé `name` a pour valeur `"Valles"` et la clé `speciality` a pour valeur `"Cardiologue"` (les deux dernières valeurs sont de type `string`). Notons que d'une manière générale, les valeurs peuvent-être de n'importe quel type. Ainsi, l'objet JSON suivant est valide :

```
1 {
2   "id": 22,
3   "name": "Valles",
4   "speciality": "Cardiologue",
5   "active": true,
6   "patients": [22, 25, 30],
7   "address": {
8     "street": "Rue de la Loi",
9     "number": 16,
10    "city": "Bruxelles"
11  }
12 }
```

On peut observer ci-dessous des valeurs de type booléen, liste (dont les valeurs sont délimitées par des crochets (`[]`)) et objet (dont les propriétés sont délimitées par des accolades (`{}`)).

Enfin, une liste de données JSON est également valide. Par exemple :

```
1 [
2   {
3     "id": 22,
4     "name": "Valles",
5     "speciality": "Cardiologist"
6   },
7   {
8     "id": 25,
9     "name": "Al-Ja'ba",
10    "speciality": "Pneumologist"
11  }
12 ]
```

Le code ci-dessus représente une liste de deux objets JSON. La liste commence et se termine par des crochets (`[]`). Au sein de cette liste, se trouve deux objets. Chaque objet est délimité par des accolades (`{}`). Les objets sont séparés par des virgules.

Chapitre 5

Mise en pratique : Express

Après avoir vu les bases théoriques d'un serveur web, mettons le en pratique avec Express.

SECTION 5.1: SERVEUR EXPRESS : LES BASES

Un serveur Express est un serveur web qui utilise le framework Express de NodeJS. Il fournit un ensemble de fonctionnalités pour les applications web et mobiles et respecte le standard RestFull. Express est donc un ensemble de libraries qui permettent de créer des applications web et des API RestFul de manière simple et rapide.

Nous l'avons vu à la section 3 un serveur Express est composé de quelques fichiers importants. Le premier d'entre eux est un fichier JSON nommé `package.json` - nous en avons présenté un exemple simple à la figure 3.1. Ce fichier contient les informations essentielles de notre serveur : son nom, la version actuellement utilisée, son créateur, une petite description, les scripts pour lancer son exécution, etc.

Parmi les éléments présents dans ce fichier, deux scripts (ou commandes) sont particulièrement importants : `build` et `dev`. Le premier `build` permet de "compiler" notre code TS en code JS (voir section ??). Le second `dev` permet de lancer notre serveur en mode développement (voir section 3.1). Lors de nos développements nous utiliserons ces deux commandes quotidiennement dans deux terminaux différents et en parallèle.

Le second fichier est le [point d'entrée de notre serveur](#) (c'est à dire le fichier qui exécute le lancement du serveur), c'est le fichier `main.ts` - nous en avons présenté un exemple simple à la figure 3.2. Reprenons ici à l'identique le contenu de ce fichier :

```
1 // file main.ts
2 import {app} from 'app';
3 // starts the server
4 app.listen(3000, () => {
5   console.log("Server is running on port 3000");
6 });
```

Ce fichier est très simple, il importe notre code et lance le serveur sur le port 3000. Regardons comment est structuré le code de notre serveur.

```

1 // file app.ts
2 import express from 'express';
3 export const app = express();
4
5 // simple hello world route
6 app.get('/', (req, res) => {
7   res.send('Hello World!');
8 });

```

Regardons en détails ce fichier. Tout d’abord nous importons le module `express` (ligne 2). Ensuite nous créons une instance de l’application Express qui est l’objet qui représente notre serveur (ligne 3). Nous définissons une route avec la méthode GET et le chemin `"/`. (lignes 6 à 8). Cette route renvoie le texte `Hello World!`. Ce fichier permet donc de définir une route simple qui est liée à la racine de notre serveur (*i.e.* `http://localhost:3000/`).

Notre serveur fonctionne donc ainsi : lorsque qu’un client accède à notre serveur à l’URL `http://localhost:3000/` avec la méthode GET, le serveur renverra le texte `Hello World!`. Vous pouvez facilement vérifier cela en démarrant votre serveur (voir section 3.1) et copiant collant l’URL `http://localhost:3000/` dans votre navigateur web et observer le résultat.

Si les deux première lignes ne demandent pas beaucoup d’explications. La définition d’une route est un peu plus complexe. Regardons les lignes 6 à 8 en détails. On voit tout d’abord que la méthode HTTP est spécifiée par l’appel sur l’objet `app` : dans notre cas nous avons `app.get(...)`. **Nous définissons donc une route HTTP de type GET.**

La méthode `get()` prend **deux paramètres** : le premier est **le chemin** avec le type `string` (ici `'/'`), le second est **une fonction**. Cette fonction prend deux paramètres `req` de type `Request` et `res` de type `Response`. `req` est l’objet qui contient les informations de la requête HTTP (comme les paramètres de la requête, le corps de la requête, etc.). `res` est l’objet qui contient les informations de la réponse HTTP.

Répétons cela pour être certain, la méthode `get` de l’objet `app` possède deux paramètres, le premier est de type `string`, le second est de type fonction. Cette fonction définit le traitement à appliquer lorsque l’URL est appelée avec la bonne méthode.

Ainsi, la ligne 7 de notre script envoie le texte `Hello World!` dans la réponse. Nous appelons donc l’objet `res` et sa méthode `send()` avec le texte `Hello World!` comme paramètre.

Enrichissons un peu notre serveur en ajoutant une autre route. Par exemple, ajoutons une route qui renvoie une liste statique des docteurs de notre hôpital. Nous allons utiliser la méthode HTTP GET car il s’agit toujours d’accéder à une ressource (la liste des docteurs). Le chemin fera références à la ressource au pluriel (*i.e.* `doctors`). Nous allons également indiquer par un code de retour que tout c’est bien passé. Le code qui définit une telle route est :

```

1 // file app.ts
2 app.get('/doctors', (req: Request, res: Response) => {
3   const doctors = [
4     {id:22,name:"Valles"},
5     {id:25,name:"Al-Ja'ba"}
6   ];
7   res.status(200).json(doctors);

```



```
8 | });
```

Listing 5.1 – Ajout d’une route pour la liste des docteurs

Ce code présente la définition d’une route référençant le chemin `/doctors`. **À chaque fois que la route est appelée par un client** une variable `doctors` est créée et initialisée avec deux objets. Ensuite, la réponse est envoyée avec le code de retour (status) 200 (qui signifie que tout c’est bien passé) et le contenu de la réponse est la liste des docteurs. La méthode `json()` de l’objet `res` permet de renvoyer une ressource sous format JSON.

Enfin, ajoutons une fonctionnalité permettant de créer un nouveau docteur. Nous allons utiliser la méthode HTTP POST car il s’agit de créer une nouvelle ressource. Le chemin sera en tout point identique à celui utilisé pour obtenir la liste. Comme il s’agit d’une requête de type POST les données passées par le client sont dans le corps de la requête (voir section 4.5.2). Nous allons également indiquer par un code de retour que tout c’est bien passé (status 200). Le code qui définit une telle route est :

```
1 // file app.ts
2 app.post('/doctors', (req: Request, res: Response) => {
3   const doctorName = req.body?.name;
4   const doctorSpeciality = req.body?.speciality;
5   if (doctorName === undefined || doctorSpeciality === undefined) {
6     res.status(400).send("Missing data");
7     return;
8   }
9   const doctorID = rand(1000); // TODO: make it better
10  const doctor: Doctor = new Doctor(doctorID, doctorName, doctorSpeciality);
11  // does something with the doctor object
12  // ...
13  // sends the new created object to the client with 201 status code
14  res.status(201).json(doctor);
15 });
```

Listing 5.2 – Ajout d’une route pour la création d’un docteur

Ce code est un peu plus complexe, et nous y reviendrons dans les prochaines sections. Toutefois, dégageons les grandes lignes. La ligne 1 indique l’on utilise la méthode HTTP POST et l’URL `/doctors`. La ligne 2 et 3 permettent de récupérer les données envoyées par le client dans le corps de la requête. Le point d’interrogation `?` permet d’indiquer que ses données peuvent-être manquante (voir section ??). Les lignes 4 à 7 permettent de vérifier que les données nécessaires à la création d’un docteur sont bien présentes et de renvoyer un code d’erreur 400 si ce n’est pas le cas. Les lignes 8 et 9 permettent de créer un nouvel objet `doctor` de type `Doctor` avec les données reçues et un ID aléatoire. Enfin, la ligne 13 permet de renvoyer la nouvelle ressource `doctors` créé avec un code de retour 201 (voir section 4.6.2).

A ce stade, le code complet de notre fichier `main.ts` est le suivant :

```

1  import express from 'express';
2  // creates an express app
3  const app = express();
4  // defines a route
5  app.get('/', (req: Request, res: Response) => {
6      res.send('Hello World!');
7  });
8  // lists the doctors
9  app.get('/doctors', (req: Request, res: Response) => {
10     const doctors = [
11         {id:22,name:"Valles"},
12         {id:25,name:"Al-Ja'ba"}
13     ];
14     res.status(200).json(doctors);
15 });
16 // creates a new doctor
17 app.post('/doctors', (req: Request, res: Response) => {
18     const doctorName = req.body?.name;
19     const doctorSpeciality = req.body?.speciality;
20     if (doctorName === undefined || doctorSpeciality === undefined) {
21         res.status(400).send("Missing data");
22         return;
23     }
24     const doctorID = rand(1000);
25     const doctor: Doctor = new Doctor(doctorID, doctorName, doctorSpeciality);
26     // does something with the doctor object
27     // ...
28     // sends the new created object to the client with 201 status code
29     res.status(201).json(doctor);
30 });
31
32 // starts the server
33 app.listen(3000, () => {
34     console.log("Server is running on port 3000");
35 });

```

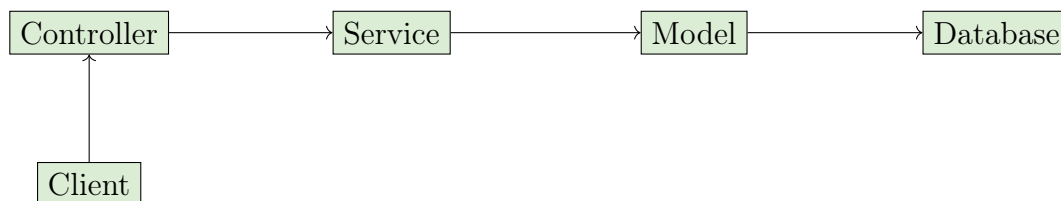
Listing 5.3 – Code complet de `main.ts`

Si à ce stade notre serveur est fonctionnel, il est encore très basique. Un serveur dispose généralement de plusieurs dizaines (voir centaines) de routes. Tout mettre dans un même fichier serait fastidieux et peu propice au travail d'équipe. Il est donc nécessaire de **structurer notre code** pour le rendre plus lisible et plus facile à maintenir. Nous allons voir comment faire cela dans la section suivante.

Pour organiser notre code, nous devons nous référer à certain modèle de conception. Notre modèle de conception fera intervenir des contrôleurs (spécifiques à chaque ressource), des services (spécifiques à des ressources ou plus généraux), des modèles (qui représentent les ressources) et une base de données (qui stocke les ressources). Nous allons voir comment mettre en place ces différents éléments dans les sections suivantes.

SECTION 5.2: ORGANISATION D'UNE APPLICATION EXPRESS

Nous l'avons vu, un serveur répond à des requêtes émises aux travers d'URL avec certains paramètres et certaines méthodes HTTP. Les routes faisant office de **point de contact** de

**FIGURE 5.1** – Organisation d'une application Express

notre serveur.

Ces points de contacts sont définis par des chemins et des méthodes HTTP. Ces points de contacts font référence à des ressources (*i.e.* des docteurs, des patients, des rendez-vous, etc.). Pour chaque ressource, nous avons donc une ou plusieurs routes qui permettent de la manipuler (voir CRUD à la section 4.5.1). De façon structurée, on regroupe les routes qui sont liées entre elles dans une même "entité" : un **controller**.

5.2.1 CONTROLLER

Un **controller** est une entité de notre modèle de conception chargée de tâches spécifiques : 1. contrôler les données envoyées par le client, 2. de les traiter en appelant les services adéquats et 3. de renvoyer une réponse au client. Un controller est lié à une ressource et est chargé de gérer les routes qui sont liées à cette ressource. Par exemple, un controller `DoctorsController` gère les routes qui sont liées à la ressource `doctors`.

On regroupe tous les contrôleurs d'un serveur au sein d'un même dossier. On nommera ce dossier **controllers**. Chaque controller est un fichier TypeScript qui contient les différentes routes liées à une ressource. Par exemple, le controller `doctors.controller.ts` contiendra les routes liées à la ressource `doctors`, le fichier `patients.controller.ts` contiendra les routes liées à la ressource `patients`.

Lorsqu'on définit une route avec Express, nous fournissons une URL (par exemple `/doctors`) ainsi qu'une fonction qui est exécutée lorsque l'URL est appelée (voir figure 5.2). Cette fonction est appelée **middleware**.

Ainsi pour chaque URL, nous avons une fonction middleware qui est exécutée. Lorsqu'on a plusieurs routes, on a donc plusieurs fonctions middleware.

Les middlewares sont des fonctions qui prennent trois paramètres : `req`, `res` et `next`. `req` est l'objet qui contient les informations de la requête HTTP, `res` est l'objet qui contient les informations de la réponse HTTP et `next` est une fonction qui permet de passer à la prochaine fonction middleware. Notons que le dernier paramètre `next` n'est pas obligatoire et ne sera pas utilisé ni présent dans le cadre de ce cours.

L'on peut résumer la définition d'une route avec Express par le code suivant :

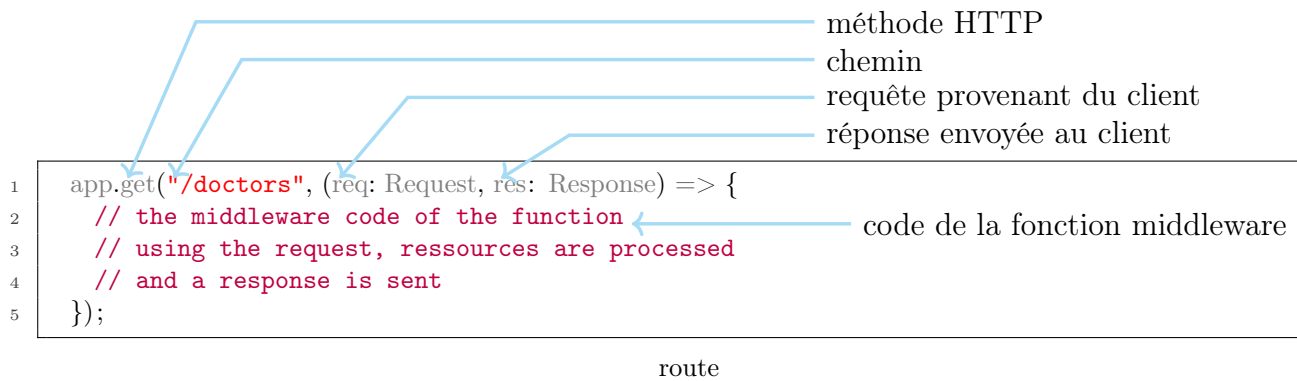


FIGURE 5.2 – Explication d'une définition de route avec Express

RÈGLES À RESPECTER LORS DE LA CRÉATION D'UN CONTROLLER

Un controller a pour premier objectif de contrôler que les informations envoyées par le client sont complètes et avec le type attendu. Cette opération est obligatoire et nécessaire dans le cadre de ce cours.

Si les données envoyées par le client ne sont pas valides, le controller arrête immédiatement son travail et renvoie au client une réponse indiquant par un status approprié que la requête n'a pas abouti et qu'elle était erronée.

Si les données fournies dans la requête sont suffisantes, le controller appelle les différents services nécessaires pour répondre à la requête et il vérifie (au besoin) qu'aucune erreur ne s'est produite et que le résultat attendu a été trouvé (dans le cas contraire, un status d'erreur doit être envoyé dans la réponse).

Une fois le traitement terminé, le serveur renvoie les potentielles données demandées et le status 200 indiquant que tout s'est bien passé.

EXEMPLE DE CONTROLLER

Illustrons ici nos règles de définition d'une route avec un exemple simple. Nous allons créer un controller pour la ressource `doctors`. Nous allons créer un fichier `doctors.controller.ts` dans le dossier `controllers` à la racine de notre projet. Ce fichier contiendra les différentes routes liées à la ressource `doctors`.

Et son utilisation au sein du fichier principal `main.ts`.

créer le fichier.

5.2.2 MODÈLE

Dans notre représentation d'une application Express, un **modèle** est une entité qui représente une ressource. Un modèle est un objet qui contient les différentes propriétés de la ressource. Par exemple, un modèle `Doctor` contiendra les propriétés `id`, `name`, `speciality`, etc. Un **modèle est un objet TS** dont **le type est**, généralement, **défini par une interface**.

Pratiquement, chaque modèle est défini dans un fichier TS dans un dossier `models` à la racine de notre projet. Chaque fichier contient une classe qui représente la ressource. Par exemple,

le modèle `doctor.model.ts` contiendra la classe `Doctor` qui représente un docteur. Ce fichier contiendra le code suivant :

```

1  export interface Doctor {
2    id: number;
3    name: string;
4    speciality : string;
5  }
```

Listing 5.4 – Exemple de modèle `doctor.model.ts`

Comme le montre le code ci-dessus, un tel fichier est généralement simple. Il ne contient qu'une déclaration d'interface présentant une liste d'attribut de l'objet `Doctor`. Le modèle `Doctor` est une **interface** qui contient les propriétés `id`, `name` et `speciality`. Ces propriétés sont respectivement de type `number`, `string` et `string`.

Une fois cette interface créée, elle peut-être utilisée dans les autres fichiers de notre application. Par exemple, dans le contrôleur `doctors.controller.ts`, nous pouvons utiliser l'interface `Doctor` pour définir les types des objets que nous manipulons. Par exemple : **respecter les normes définie dans le premier controller a propos des infos manquantes.**

```

1  import { Doctor } from '../models/doctor.model';
2  // ...
3  // a route to update a doctor
4  app.put('/doctors', (req: Request, res: Response) => {
5    // extract information from request body - it is a PUT request
6    const doctor: Doctor = {
7      id: req.body?.id,
8      name: req.body?.name;
9      speciality : req.body?.speciality ,
10   };
11   // ...
12 });
```

5.2.3 SERVICE

Dans notre modèle de conception, un **service** est une entité qui est chargée de gérer certaines données. Un service est chargé de récupérer les données, de les traiter et de les envoyer vers le programme appelant. Un service est donc **une couche d'abstraction** entre le contrôleur et un autre élément de notre serveur (par exemple : la base de données).

Cette architecture est intéressante dans un environnement de travail évolutif. En effet, si l'on change de base de données, il suffit de changer le service sans toucher au contrôleur.

De la même façon que les contrôleurs, les services sont stockés sous la forme de fichiers TypeScript dans un dossier `services`. Chaque service est un fichier qui contient les différentes fonctions qui permettent de récupérer, de traiter et de renvoyer les données. Par exemple, le service `doctors.service.ts` contiendra les fonctions qui permettent de récupérer, de traiter et de renvoyer les données liées à la ressource `doctors` depuis et vers la base de données.

Un service attaché à une ressource est généralement une classe qui contient des méthodes statiques répondant au minimum au CRUD (voir section 4.5.1). Par exemple, un service `DoctorsService` contiendra les méthodes `create`, `getById`, `list`, `update` et `delete`. Chacune de ces méthodes contacte la base de données pour effectuer les opérations nécessaires. Par exemple, la méthode `create` contacte la base de données pour créer un nouveau docteur.

Cela s'effectue concrètement par l'intermédiaire d'une requête SQL (voir plus loin section 5.2.4).

Un service peut également être utilisé pour effectuer des opérations plus complexes. Par exemple, un service `DoctorsAppointmentsService` peut être utilisé pour gérer les rendez-vous. Il peut contenir des méthodes pour vérifier la disponibilité d'un docteur, pour créer un nouveau rendez-vous, ou encore déplacer ou annuler un rendez-vous existant, etc.

Certains service sont utilisé pour des opérations plus générales. Par exemple, un service `AuthService` peut être utilisé pour gérer l'authentification des utilisateurs. Il peut contenir des méthodes pour vérifier si un utilisateur est authentifié, pour vérifier si un utilisateur est autorisé à effectuer une action, etc.

Un autre service classique est le logger. Un `logger` est un composant de nombreuses installation informatique, son role est d'enregistrer toutes actions effectuées par le système et ses clients (les *logs*). Il permet de garder une trace de toutes les actions effectuées par le système. Il est donc très utile pour le débogage, pour la sécurité, pour l'analyse des performances, etc. Nous reviendrons sur ce point dans la section 6.

RÈGLES À RESPECTER LORS DE LA DÉFINITION D'UN SERVICE

EXEMPLE DE SERVICE

Voici un exemple de service simple lié à la ressource `doctors`. Nous allons créer un fichier `doctors.service.ts` dans le dossier `services` à la racine de notre projet. Ce fichier contiendra les différentes fonctions qui permettent de récupérer, de traiter et de renvoyer les données liées à la ressource `doctors` depuis la base de données.

```

1  import { Doctor } from '../models/doctor.model';
2  import { db } from '../database';
3
4  export class DoctorsService {
5      // create a new doctor
6      static create(name: string, speciality: string): boolean {
7          try{
8              db.prepare("INSERT INTO doctors (name, speciality) VALUES (?, ?)");
9              db.run(name, speciality);
10         } catch(error: Error) {
11             console.error(error);
12             return false;
13         }
14         return true;
15     }
16
17     // get a doctor by its ID
18     static getById(id: number): Doctor | null {
19         db.prepare("SELECT * FROM doctors WHERE id = ?");
20         const data = db.get(id);
21         if (data === undefined) {
22             // data not found
23             return null;
24         }
25         const doctor: Doctor = {
26             id: doctor.id,
27             name: doctor.name,
28             speciality: doctor.speciality
29         };
30         return doctor;
31     }
32
33     // list all doctors
34     static list(): Doctor[] {
35         db.prepare("SELECT * FROM doctors")
36         const data = db.all();
37         const doctors: Doctor[] = []; // the response to send
38         for (const doctor of data) {
39             doctors.push({
40                 id: doctor.id,
41                 name: doctor.name,
42                 speciality: doctor.speciality
43             });
44         }
45         return doctors;
46     }
47 }

```

Nous observons plusieurs choses dans le code ci-dessus. Tout d'abord, le fichier commence par importer les bibliothèques nécessaires. Il s'agit ici du modèle et des éléments permettant de se

connecter à la base de données.

Le service en lui-même est une `class` dont le nom est `DoctorsService`. Cette classe est exportée, et un autre fichier peut donc l'importer et l'utiliser. La classe contient trois méthodes statiques : `create`, `getById` et `list`. La méthode `create` permet de créer un nouveau docteur dans la base de données. La méthode `getById` permet de récupérer un docteur par son ID. Et `list` permet d'accéder à la liste de tous les docteurs. Toutes trois exécutent une requête SQL sur la base de données en utilisant la variable `db` qui représente une connection vers la base de données SQL.

Ces méthodes appellent la méthode `prepare` de la variable `db` qui permet de préparer une requête à envoyer. Le paramètre de la fonction est une ligne de commande SQL valide (voir cours de DB) où les valeurs à insérer sont remplacées par des points d'interrogation `?`.

Toutefois, une fois la méthode `prepare` appelée, **il est nécessaire d'appeler une autre méthode de façon à exécuter la commande SQL. La méthode à appeler dépend de l'action voulue sur la base de donnée :**

- `run` : pour les requêtes de type INSERT, UPDATE et DELETE.
- `get` : pour les requêtes de type SELECT qui ne renvoient, au plus, qu'une seule ligne.
- `all` : pour les requêtes de type SELECT qui renvoient plusieurs lignes.

5.2.4 LE SERVICE D'ACCÈS BASE DE DONNÉES

Un avantage de l'architecture que nous avons mis en place est qu'il est facile d'utiliser un service pour abstraire des opérations telles que l'accès à une base de données. Lors du développement d'un projet, il peut s'avérer indispensable de faire évoluer les technologies utilisées. Par exemple, si l'on décide de passer d'une base de données SQL à un autre type de base de données il sera alors nécessaire de modifier le code de notre serveur.

Il est donc important de choisir un design d'architecture logiciel permettant d'effectuer ces évolutions de manière simple et rapide. Si nous utilisons un service `SQLDatabaseService` pour gérer l'accès à la base de données, il suffira de modifier ce service pour que notre serveur fonctionne avec la nouvelle base de données. Nous créerons alors un nouveau service, par exemple `MongoDatabaseService` pour gérer l'accès à la base de données MongoDB. Les changements nécessaires seront alors minimes dans le reste du code : modifier l'`import` dans les différents fichiers qui utilisent le service de base de données. Et voilà ! Notre serveur fonctionne avec une nouvelle base de données.

Regardons comment mettre en place un tel service. Dans le cadre de ce cours, nous n'utiliserons que la base de données SQL. Il ne sera pas question d'évolution ici. Nous allons créer un fichier `doctors.service.ts` dans le dossier `services` à la racine de notre projet. Ce fichier contiendra les différentes opérations possibles sur la table relative à la ressource `doctors`. Un exemple d'un tel service pourrait être :


```

1  import sqlite3 from 'sqlite3';
2  import { Doctor } from '../models/doctor.model';
3
4  export class SQLDatabaseService {
5      private static db: sqlite3.Database;
6      static connect(): void {
7          this.db = new sqlite3.Database('our.db.name');
8      }
9      static disconnect(): void {
10         this.db.close();
11     }
12     /** TODO make documentation */
13     static createDoctor(name: string, speciality: string): boolean {
14         try{
15             this.db.prepare("INSERT INTO doctors (name, speciality) VALUES (?, ?)");
16             this.db.run(name, speciality);
17         } catch(error: Error) {
18             console.error(error);
19             return false;
20         }
21         return true;
22     }
23     static getDoctorById(id: number): Doctor | null {
24         this.db.prepare("SELECT * FROM doctors WHERE id = ?");
25         const data = this.db.get(id);
26         if (data === undefined) {
27             // data not found
28             return null;
29         }
30         const doctor: Doctor = {
31             id: doctor.id,
32             name: doctor.name,
33             speciality: doctor.speciality
34         };
35         return doctor;
36     }
37     static listDoctors(): Doctor[] {
38         this.db.prepare("SELECT * FROM doctors")
39         const data = this.db.all();
40         const doctors: Doctor[] = []; // the response to send
41         for (const doctor of data) {
42             doctors.push({
43                 id: doctor.id,
44                 name: doctor.name,
45                 speciality: doctor.speciality
46             });
47         }
48         return doctors;
49     }
50 }

```

Listing 5.5 – Exemple de service d'accès à la table doctors

5.2.5 LE SERVICE LOGGER

SECTION 5.3: LES DTO

SECTION 5.4: CONVENTION DE NOMMAGE

Dans le cadre de ce cours, les routes que vous créez devront respecter les conventions suivantes :

- les routes doivent être écrite en anglais ;
- les routes doivent être en minuscule ;
- le premier mot de la route doit être le nom de la ressource ;
- le second mot de la route doit faire référence à l'action à effectuer sur la ressource ou au paramètre ;

SECTION 5.5: EN BREF

A la fin de ce chapitre, votre serveur devrait se composer de :

- un fichier `package.json` contenant tous les informations générales de votre serveur ainsi que les dépendances nécessaires, les scripts de lancement, etc. (voir code 3.1),
- un fichier `main.ts` qui contient le code de lancement du serveur et les différentes routes (voir code 5.3),
- un dossier `controllers` qui contient les différents contrôleurs de votre serveur (voir section 5.2.1),
- un dossier `models` qui contient les différents modèles de votre serveur (voir section 5.2.2),
- un dossier `services` qui contient les différents services de votre serveur (voir section 5.2.3).

SECTION 5.6: EXERCICES

SECTION 5.7: CORRECTION DES EXERCICES

Chapitre 6

Deux services particuliers

SECTION 6.1: LOGGER

Dans de nombreux système informatiques il est nécessaire de garder une trace des événements qui se produisent. Cela peut être utile pour : déboguer un programme, pour analyser le comportement d'un système, pour vérifier que les règles de sécurité sont respectées, pour retrouver la cause d'une panne, etc.

L'outil le plus couramment utilisé pour cela est le *logger*. Un logger est un programme qui écrit des messages. Ces messages peuvent être de différents niveaux. Dans le cadre de ce cours, nous en retiendrons 3 : *debug*, *info*, *error*. Chaque message est associé à un niveau de criticité. Par exemple, un message de niveau *debug* est un message qui ne sera pas vu par l'utilisateur final, mais qui est utile pour le développeur. Un message de niveau *error* est un message qui indique qu'une erreur s'est produite. Le niveau *info* est un niveau intermédiaire et enregistre les actions effectuées par l'utilisateur final.

Identifier quel message va sur quel niveau est une compétence de l'informaticien·ne. Il n'existe pas de règle absolue et selon le contexte, l'entreprise, etc. les nombre de niveaux peut varier et leur signification aussi.

Dans le cadre de ce cours, nous utiliserons les règles suivantes :

- *debug* : tout message non pertinent lors de l'utilisation normale du programme, mais qui peut être utile pour déboguer le programme. Par exemple :
 - tout message permettant de vérifier la valeur d'une variable,
 - tout message permettant de vérifier le bon déroulement d'une boucle, d'une fonction,
 - etc.
- *info* : message d'information gardant une traçabilité des actions effectuées par l'utilisateur final. Par exemple :
 - un message indiquant qu'un utilisateur s'est connecté,
 - un message indiquant qu'un utilisateur a emprunter une route particulières,
- *error* : message d'erreur permettant d'informer sur toute tâche qui n'aurait pas été réalisée avec succès
 - un message indiquant qu'une erreur s'est produite lors de la connexion d'un utilisateur,
 - un message indiquant qu'une erreur s'est produite lors de l'accès à une ressource particulière (exemple : `id` invalide, route innexistante, etc.).

Il est important de respecter les conventions suivantes :

- aucune information personnelle ne doit être enregistrée (ni login, ni mot de passe, ni adresse email, etc.),
- les messages doivent être clairs et concis,

- les messages doivent contenir toute l'informations nécessaire pour comprendre ce qui s'est passé.

SECTION 6.2: LOGGER AS A SERVICE

Dans le cadre de notre cours, le logger sera un service. Ce service sera utilisé par d'autres éléments de notre serveur (controller et service) pour enregistrer des messages. Notre logger sera implémenté comme une classe statique. Cela signifie que nous n'aurons pas besoin d'instancier un objet de cette classe pour l'utiliser (voir cours de APPO). Nous pourrions appeler directement les méthodes de la classe.

```

1 // fichier logger.service.ts
2 class Logger {
3   static void debug(string message) {
4     System.out.println("DEBUG: " + message);
5   }
6
7   static void info(string message) {
8     System.out.println("INFO: " + message);
9   }
10
11  static void error(string message) {
12    System.out.println("ERROR: " + message);
13  }
14 }
```

Listing 6.1 – un exemple de logger

Le code ci-dessus présente un logger très simple. Il contient 3 méthodes : `debug`, `info` et `error`. Chaque méthode prend en paramètre un message et l'affiche sur la console. Le message est précédé du niveau du message. Ce type de service est très simple et sera amené à être amélioré.

Ce logger s'utilise de la manière suivante :

```

1 // fichier doctors.controller.ts
2 import { Logger } from '../services/logger.service';
3
4 // get all the doctors
5 DoctorsController.get('/', (req: Request, res: Response) => {
6   Logger.info('GET /doctors');
7   res.json(doctors).status(200);
8 });
9
10 // get a specific doctor
11 DoctorsController.get('/:id', (req: Request, res: Response) => {
12   Logger.info('[GET] /doctors/:id');
13   let doctor: Doctor | undefined = undefined;
14   // extract the id from Request and verify that it is a number
15   const id = parseInt(req.params.id);
16   if (isNaN(id)) {
17     Logger.error(`Invalid id ${req.params.id}`);
18     res.status(400).send('Invalid id');
19   } else {
20     // .. code to find the doctor
21   }
```

22 | });

Listing 6.2 – utilisation du logger

Vous noterez que nous avons importé le service `Logger` dans le fichier `doctors.controller.ts`. Nous avons ensuite utilisé la méthode `info()` pour enregistrer un message. Ce message est enregistré dans la console (par définition de la fonction `info`). Notez bien que nous n'avons pas fait de `new Logger()` avant d'utiliser la méthode `info`. Cela est possible car la méthode `info` est statique.